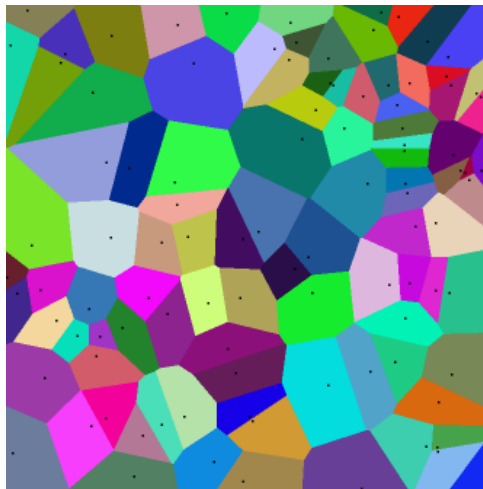


SAÉ S6 : Diagrammes de Voronoï

Phase 2 : Développement avec IA

Équipe : Moilim ABDALLAH, Syphax BENCHALAL, Marwan CHKIRIBA, Mehdi CHOUGRANI, Robert GRIFFATON, Stéphane GUENOUNOU

Enseignants : J-C. Dubacq, M. Mayero, H. Zargayouna



14 juin 2026

Table des matières

Introduction	2
1 Claude 4.6 Sonnet	3
1.1 Prompt initial utilisé	3
1.2 Journal des corrections et historique	4
1.2.1 Prompts de correction et d'ajustements	4
1.3 Tests logiciels	5
2 ChatGPT 3.5	7
2.1 Prompt initial utilisé	7
2.2 Journal des corrections et historique	7
2.2.1 Prompts de correction et d'ajustements	7
2.3 Tests logiciels	7
3 Gemini 3.1 Pro	9
3.1 Prompt initial utilisé	9
3.2 Journal des corrections et historique	9
3.2.1 Prompts de correction et d'ajustements	9
3.3 Tests logiciels	10
4 Kimi K2.5	12
4.1 Prompt initial utilisé	12
4.2 Journal des corrections et historique	12
4.2.1 Aucun correctif nécessaire	12
4.3 Tests logiciels	12
5 Tableau comparatif : Humain vs IA	14
5.1 Comparaison multi-critères	14
5.1.1 Efficacité et productivité	14
5.1.2 Qualité technique du livrable	15
5.1.3 Nature des difficultés rencontrées	16
5.2 Analyse de l'efficacité	17
Conclusion	18

Introduction

L'objectif de cette seconde phase de SAÉ est de mesurer l'impact et l'efficacité des intelligences artificielles génératrices de code dans notre développement. Après une première phase de développement sans IA, nous avons confronté quatre modèles d'IA (ChatGPT, Claude, Gemini, Kimi) au même cahier des charges strict : création d'une application de calcul de diagrammes de Voronoï avec interface Tkinter, export SVG/Image et respect des principes SOLID et Clean Code.

Chapitre 1

Claude 4.6 Sonnet

1.1 Prompt initial utilisé

Nous avons fait en sorte de bien documenter l'IA afin qu'elle produise le travail le plus qualitatif possible, d'où la longueur du prompt :

```
Role : Tu es un expert developpeur python specialise en geometrie
      algorithmique et
conception d'interfaces graphiques.
Objectif : L objectif est le developpement d une application de
          generation de diagrammes de
Voronoi en Python.
Contexte : Ce projet s'inscrit dans le cadre d'une SAE de 6eme
          semestre en BUT
Informatique. Apres une phase de developpement manuel, nous entrons
          dans la phase 2 : la
realisation assistee par IA pour comparer les performances de
          differents modeles. L'objectif
est de produire une application repondant a un cahier des charges
          strict tout en respectant
les bonnes pratiques de developpement.
1. Mission :
Developper une application Python complete qui :
- Lit un fichier d'entree contenant une liste de points (
  coordonnees x,y au format
nombre,nombre, une par ligne).
- Calcule le diagramme de Voronoi associe.
- Affiche le resultat via une interface graphique realisee avec
  Tkinter.
- Permet l'export du diagramme au format SVG ou sous forme d'image.
2. Exigences de qualite et methodologie :
Tu dois agir comme un expert developpeur senior et appliquer
          strictement les principes
suivants :
- TDD : Presente le code comme s il avait ete concu en TDD. Fournis
          une suite de
tests complets avant ou parallelement au code logique.
- Principe KISS : Chaque ligne de code doit avoir une raison d'etre
          .
```

```

- Principes SOLID : L'architecture doit etre modulaire, extensible
  et respecter la
responsabilite unique SRP.
- Clean Code :
- Nommage explicite des variables et fonctions (en anglais).
- Fonctions courtes et specialisees.
- Gestion robuste des exceptions (donnees invalides).
- Aucun "Magic Number" : Utilise des constantes nommees.
3. Specifications des tests : L'application doit inclure une serie
  de tests avec Pytest.
- Structure des tests : Utilise le pattern Arrange / Act / Assert (
  AAA).
- Nommage des tests : Chaque test doit suivre strictement le format
  : Should_given
4. Architecture et design patterns :
- Choix de l'algorithme : Tu es libre de choisir l'algorithme le
  plus pertinent et le plus
optimise pour le calcul de Voronoi (ex: Fortune, Incremental, ou
  via une bibliotheque
optimisee). Justifie brievement ce choix.
- Design patterns : Utilise des patterns s ils sont pertinents (ex:
  strategy, factory)
Explique l'interet de leur utilisation dans ce contexte.
- Anti-patterns a eviter : Interdiction de produire une "God Class
  ", du code
"Spaghetti" etc...
5. Livrables attendus :
1. L'arborescence complete du projet.
2. Le code source Python commente et type.
3. Le fichier de tests Pytest complet.
4. Un court guide de lancement (installation des dependances,
  execution de l'app et
des tests).

```

Listing 1.1 – Prompt de cadrage expert

1.2 Journal des corrections et historique

Le développement avec Claude 4.6 Sonnet a duré 1h45. Bien que l'IA ait produit une structure solide rapidement, le temps restant a été consacré à la correction des tests unitaires qui ne passaient pas initialement et au peaufinement de l'application.

1.2.1 Prompts de correction et d'ajustements

— Correction 1 : pytest non reconnu sous PowerShell

À l'exécution de `pytest tests/ -v`, PowerShell retournait une erreur `CommandNotFoundExcepti`. Claude a expliqué qu'il fallait passer par le module Python plutôt que par la commande directe, et a mis à jour le README en conséquence avec la commande `python -m pytest tests/ -v`. Il a également précisé que les dossiers `src{core,ui,io,export}`

initialement vides étaient normaux car les fichiers `__init__.py` suffisaient à créer les packages Python.

— **Correction 2 : échec d'un test sur le namespace SVG**

Le test `test_Should_produce_valid_xml_given_valid_diagram` échouait avec l'assertion `assert '{http://www.w3.org/2000/svg}svg' == 'svg'`, car `ElementTree` préfixe automatiquement le tag racine avec le namespace XML complet. Claude a corrigé le test en acceptant les deux formes de tag :

```
SVG_NAMESPACE = "http://www.w3.org/2000/svg"
assert root.tag in ("svg", f"{{{SVG_NAMESPACE}}}svg")
```

— **Correction 3 : absence du fichier d'exemple de points**

Le dossier `resources/` était vide alors que l'IA nous assurait qu'il était bien dans l'arborescence. À la demande «*il n'y a pas de fichier .txt dans mon projet, où l'ajouter ?*», Claude a indiqué le bon emplacement où il fallait mettre le dossier finalement(`src/resources/sample_points.txt`) et a généré un fichier d'exemple conforme au format attendu avec des coordonnées entières et décimales.

1.3 Tests logiciels

L'application comporte 45 tests unitaires répartis en 4 fichiers, tous structurés selon le pattern AAA et nommés suivant la convention `Should_<résultat>_given_<contexte>`.

Fichier	Nb tests	Périmètre
<code>test_models.py</code>	13	Point, BoundingBox
<code>test_point_file_reader.py</code>	11	Parsing, erreurs de format
<code>test_voronoi_engine.py</code>	11	Calcul, doublons, cas limites
<code>test_exporters.py</code>	10	SVG valide, PNG valide, taille fichier

TABLE 1.1 – Répartition des tests générés par Claude

Deux exemples représentatifs :

```
def test_Should_return_correct_points_given_valid_file(
    self, reader, valid_points_file):
    # Act
    points = reader.read(valid_points_file)
    # Assert
    assert len(points) == 5
    assert points[0] == Point(2.0, 4.0)
    assert points[1] == Point(5.3, 4.5)
```

Listing 1.2 – Test du parseur de fichier (AAA)

```
def
test_Should_raise_InsufficientPointsError_given_only_two_distinct_points
(
    self, engine):
    # Arrange
    points = [Point(0, 0), Point(1, 1)]
    # Act / Assert
```

```
with pytest.raises(InsufficientPointsError):  
    engine.compute(points)
```

Listing 1.3 – Test du moteur de calcul — gestion des erreurs

Le résultat final des tests après corrections est 45 passed en environ 0.3 secondes.

Chapitre 2

ChatGPT 3.5

2.1 Prompt initial utilisé

Nous avons utilisé le même prompt initial afin de comparer équitablement les différents modèles.

2.2 Journal des corrections et historique

2.2.1 Prompts de correction et d'ajustements

Trois correctifs ont été nécessaires après la génération initiale :

- **Correction 1 : absence de fichier de points**

L'application a été livrée sans fichier d'exemple comme dans l'application faite par Claude. Suite à la demande *«il est où le fichier d'entrée, si il n'y en a pas génère en un»*, ChatGPT a créé le fichier `points_sample.txt` contenant 12 points de coordonnées x et y, directement avec le bouton *Load Points* de l'interface.

- **Correction 2 : absence de couleurs**

Le diagramme initial s'affichait en noir et blanc, et nous voulions ajouter de la couleur. À la demande *«ajoute de la couleur, genre différentes couleurs comme les vrais diagrammes de Voronoï vu sur Google, et améliore l'esthétique»*, ChatGPT a par la suite fait un diagramme de Voronoï coloré. Les exports SVG et PNG ont également été mis à jour.

- **Correction 3 : tests non exécutables dans l'environnement de ChatGPT**

ChatGPT a signalé que `pytest` n'était pas installé dans son propre environnement, ce qui a empêché la validation des tests. La compilation Python fonctionnait néanmoins grâce à la commande `python3 -m compileall`. Les tests ont pu être exécutés localement sans erreur après installation des dépendances.

2.3 Tests logiciels

Les tests suivent la convention de nommage `Should_given` et le pattern AAA (Arrange / Act / Assert). ChatGPT a également ajouté des tests spécifiques pour l'ajout des couleurs suite à la correction 2 :

```

def test_Should_given_sameIndex_returnSameColor() -> None:
    # Arrange
    index = 7
    # Act
    first_color = get_cell_color(index)
    second_color = get_cell_color(index)
    # Assert
    assert first_color == second_color

def test_Should_given_differentIndexes_returnDifferentColors() ->
None:
    # Arrange
    first_index = 1
    second_index = 2
    # Act
    first_color = get_cell_color(first_index)
    second_color = get_cell_color(second_index)
    # Assert
    assert first_color != second_color
    assert first_color.startswith("#") and len(first_color) == 7
    assert second_color.startswith("#") and len(second_color) == 7

```

Listing 2.1 – Tests de palette générés pour l’ajout des couleurs

Chapitre 3

Gemini 3.1 Pro

3.1 Prompt initial utilisé

Le même prompt a été utilisé comme base. Nous avons utilisé Gemini en mode 3.1 Pro, la variante la plus puissante disponible actuellement, afin d'obtenir la meilleure application possible.

3.2 Journal des corrections et historique

Le développement avec Gemini 3.1 Pro a nécessité plusieurs correctifs après la génération initiale pour obtenir un résultat vraiment propre. Il se comporte plus comme un assistant qu'il faut guider contrairement à d'autres LLM.

3.2.1 Prompts de correction et d'ajustements

- **Correction 1 : points germes absents de l'affichage**

Le diagramme initial affichait uniquement les arêtes, sans les points d'origine. À la demande *«on ne voit pas du tout les points d'origine sur l'écran»*, Gemini a ajouté le rendu des germes en rouge dans l'interface Tkinter, dans l'export SVG (`<circle cx=... cy=... r="3" fill="red"/>`) et dans l'export image (via `ImageDraw.ellipse`).

- **Correction 2 : tests cassés après ajout des points**

Les tests de `test_export_strategy.py` échouaient après la correction 1 car le mock de `VoronoiResult` ne contenait pas de champ `points`. Gemini a mis à jour `mock_voronoi` pour inclure une liste de points valide, alignant les tests avec le nouveau modèle de données.

- **Correction 3 : cellules de bord non fermées**

Les points situés aux extrémités du diagramme apparaissaient sans zone. Ces points(germes) ne possédaient pas de délimitation entre eux. Gemini a introduit une bounding box passée à `VoronoiGenerator.generate()` et utilisé `shapely.geometry.box` pour cliper les polygones, garantissant que toutes les cellules soient fermées aux limites de l'affichage.

J ai remarque un bug important : les points qui sont sur les bords du diagramme ne sont pas separees par des traits. Ils ont l air de flotter sans zone definie en etant plus ou moins proche sans etre dans une zone..

Dans le diagramme de Voronoi, chaque point doit etre dans une cellule. Je pense que le code oublie de tracer les limites ou qu il ne ferme pas les cellules sur les bords de l ecran.

Est-ce que tu peux corriger le code pour :

Tracer les separations entre TOUS les points, meme ceux a l exterieur.

Utiliser une Bounding Box pour que les cellules des bords soient bien fermees aux limites de l affichage.

Verifier que ca s affiche bien dans Tkinter et que c est aussi correct dans l export SVG.

Listing 3.1 – Prompt utilisé

3.3 Tests logiciels

Les tests suivent le pattern AAA et la convention `test_should_<résultat>_given_<condition>`. Ils couvrent trois modules, avec un usage des mocks (`unittest.mock`) pour isoler les tests du système de fichiers.

Fichier	Périmètre
<code>test_voronoi_generator.py</code>	Calcul valide, trop peu de points, colinéaires, bbox custom
<code>test_export_strategy.py</code>	Export SVG valide, export PNG valide
<code>test_file_reader.py</code>	Lecture valide, fichier absent, format invalide, lignes vides

TABLE 3.1 – Répartition des tests générés par Gemini 3.1 Pro

Deux exemples représentatifs :

```
def test_should_raise_value_error_given_collinear_points():
    # Arrange
    points = [Point(0, 0), Point(1, 1), Point(2, 2)]
    # Act & Assert
    with pytest.raises(ValueError,
        match="Could not generate Voronoi diagram."):
        VoronoiGenerator.generate(points)
```

Listing 3.2 – Test du générateur — points colinéaires

```
def test_should_raise_value_error_given_invalid_coordinate_format():
    :
    # Arrange
    file_content = "10.5,20.0\ninvalid,40.1"
    # Act & Assert
    with patch("builtins.open", mock_open(read_data=file_content)):
```

```
with pytest.raises(ValueError,
    match="Invalid coordinate format at line 2."):
    FileReader.read_points("dummy_path.txt")
```

Listing 3.3 – Test du parseur avec mock — format invalide

Chapitre 4

Kimi K2.5

4.1 Prompt initial utilisé

Le même prompt que Claude et ChatGPT et Gemini a été utilisé, afin d'avoir une bonne comparaison.

Contrainte particulière : Kimi K2.5 étant utilisé en version gratuite, aucune requête de correction supplémentaire n'était possible sans passer à une offre payante. Cette contrainte a imposé une rédaction de prompt très précis et calibré.

4.2 Journal des corrections et historique

Le développement avec Kimi K2.5 a duré 0 minute de corrections. L'IA a produit dès la première réponse un livrable entièrement fonctionnel, sans aucun correctif supplémentaire. La contrainte de ne pouvoir faire qu'un seul prompt (impossibilité de relancer des requêtes en version gratuite) a rendu cette efficacité indispensable — et Kimi l'a parfaitement gérée.

4.2.1 Aucun correctif nécessaire

Contrairement aux autres modèles testés, aucun échange correctif n'a eu lieu avec Kimi K2.5. Le projet livré était directement exécutable, les tests passaient, et l'arborescence incluait même un dossier `examples/` avec des fichiers de sortie préexistants (`output.svg`, `output.png`).

4.3 Tests logiciels

L'application embarque 5 fichiers de tests structurés selon le pattern AAA et la convention `test_should_<résultat>_given_<condition>`. Une particularité est la présence de tests d'intégration (`test_integration.py`) qui vérifient au-delà des simples tests unitaires.

Deux exemples représentatifs illustrant la richesse des cas de test :

```
def test_should_complete_workflow_given_valid_input_file(self) ->
    None:
        # Arrange - Create input file
        content = "# Test points\n0,0\n2,0\n1,1.732\n"
        # ... creation fichier temporaire ...
```

Fichier	Périmètre
test_models.py	Point, BoundingBox, VoronoiCell, VoronoiDiagram
test_file_parser.py	Parsing, commentaires, espaces, notation scientifique
test_voronoi_calculator.py	Calcul, cas limites, clipping, 100 points aléatoires
test_exporters.py	SVG, PNG/JPEG, dimensions, Factory, formats inconnus
test_integration.py	Flux complet, multi-format, grand jeu de données

TABLE 4.1 – Répartition des tests générés par Kimi K2.5

```
# Act - Parse, calculate, export
points = PointFileParser().parse(input_path)
diagram = VoronoiCalculator().calculate(points)
exporter = ExporterFactory.create_exporter(output_path)
exporter.export(diagram, output_path)
# Assert
assert Path(output_path).exists()
assert Path(output_path).stat().st_size > 0
```

Listing 4.1 – Test d'intégration bout en bout

```
def test_should_parse_file_given_scientific_notation(self) -> None:
    # Arrange
    parser = PointFileParser()
    content = "1.5e2,2.5e-1\n3.0E3,4.0"
    # ... creation fichier temporaire ...
    # Act
    points = parser.parse(temp_path)
    # Assert
    assert len(points) == 2
    assert points[0].x == 150.0
    assert points[0].y == 0.25
```

Listing 4.2 – Test du parseur — notation scientifique

Chapitre 5

Tableau comparatif : Humain vs IA

5.1 Comparaison multi-critères

Les tableaux suivants comparent le développement humain (Phase 1) et les quatre IA génératives testées (Phase 2) selon trois critères principaux : efficacité, qualité technique et nature des difficultés rencontrées.

5.1.1 Efficacité et productivité

Critère	Humain	Claude	ChatGPT	Gemini	Kimi K2.5
Temps total	10h00	1h45	~2h00	~2h00	<30 min
Nb. de prompts	—	~4	~3	6	1
Nb. de correctifs	En continu	3	3	5	0
Temps de débogage et amélioration	Faible	~30 min	~30 min	~1h	0
Fonctionnel au 1 ^{er} essai	Oui (partiel)	Non	Non	Non	Oui

TABLE 5.1 – Comparaison de l’efficacité et de la productivité

5.1.2 Qualité technique du livrable

Critère		Humain	Claude	ChatGPT	Gemini	Kimi K2.5
Architecture modulaire		Basique	Oui	Oui	Oui	Oui
Respect des principes SOLID		Partiel	Oui	Oui	Oui	Oui
Tests unitaires (Py-test)		3	45 tests	10 tests	13 tests	97 tests
Tests d'intégration		Non	Non	Non	Non	Oui
Gestion des exceptions		Basique	Robuste	Robuste	Robuste	Robuste
Export SVG + Image		Oui	Oui	Oui	Oui	Oui
Fichier d'exemple fourni		Oui	Non	Non	Oui	Oui

TABLE 5.2 – Comparaison de la qualité technique des livrables

5.1.3 Nature des difficultés rencontrées

Dimension	Développement Humain	Développement assisté IA
Source des erreurs	Erreurs logiques progressives, découvertes en testant manuellement l'application	Erreurs souvent cachées : code syntaxiquement correct mais comportement erroné (mauvaise bounding box, tests cassés)
Nature des correctifs	Simple et ciblés : une erreur = une ligne ou une fonction à corriger, directement localisable	Complexes : un correctif visuel peut casser des tests existants, nécessitant plusieurs correctifs
Compréhension du code	Totale : le développeur est l'auteur de chaque ligne et maîtrise l'ensemble de la base de code	Partielle : le code généré est souvent bien structuré mais difficile à modifier sans risque de bugs si on ne comprend pas tout
Débogage	Intuitif et rapide : le développeur sait où chercher en cas de bug	Les bugs complexes entre modules (UI, export, tests) nécessitent de comprendre du code qu'on n'a pas écrit ou de dépendre à nouveau de l'IA
Gestion des exigences	Implémentées progressivement selon les priorités, parfois incomplètes	Toutes les exigences du prompt sont traitées d'un coup, mais certaines peuvent être mal interprétées ou partiellement respectées
Courbe d'apprentissage	Forte : chaque fonctionnalité représente une opportunité d'apprentissage	Faible à court terme : le code est produit rapidement mais les compétences techniques ne progressent pas beaucoup
Reproductibilité	Totale : le même développeur peut recréer et faire évoluer le projet librement	Limitée : une nouvelle session IA peut produire un code différent, incompatible avec la version précédente
Maintenabilité future	Bonne si bonnes pratiques appliquées, mais souvent limitée par les mauvaises habitudes initialement prises	Variable : élevée si le code suit SOLID/Clean Code (Claude, Kimi), mais risquée si l'équipe ne maîtrise pas le code généré

TABLE 5.3 – Analyse comparée de la nature des difficultés Humain vs IA

5.2 Analyse de l'efficacité

Le gain de temps est indéniable : là où le développement humain a nécessité 5 heures pour une application minimaliste sans tests, les IA ont produit en moins de 2 heures des architectures complètes respectant SOLID, TDD et Clean Code. Kimi K2.5 pousse ce résultat à l'extrême avec un livrable fonctionnel en une seule requête, sans aucune correction.

Cependant, ce gain de temps ne doit pas masquer une réalité importante : le temps humain est qualitativement différent du temps IA. Une heure passée à déboguer du code que l'on n'a pas écrit (Gemini, 5 correctifs) est bien moins formatrice qu'une heure passée à implémenter soi-même un algorithme. De plus, les correctifs humains sont généralement simples et ciblés (corriger une formule, ajouter une condition), tandis que les correctifs IA déclenchent souvent des régressions dans d'autres modules.

Conclusion

Cette phase 2 de la SAÉ nous a permis de confronter concrètement le développement humain à quatre IA génératives sur un cahier des charges identique et exigeant. Les résultats sont à la fois encourageants et nuancés.

L'IA : un gain de productivité énorme. Toutes les IA testées ont produit des architectures que cinq heures de développement humain n'avaient pas permis d'atteindre : typage Python complet, principes SOLID respectés, suites de tests structurées en AAA. Le gain de temps dépasse 60% sur l'ensemble des modèles, et Kimi K2.5 démontre qu'il est possible d'obtenir un projet complet en un seul prompt bien formulé.

Une fiabilité qui varie selon l'IA choisie. Les quatre IA n'ont pas affiché le même niveau de fiabilité. Kimi K2.5 (0 correction) et Claude (3 corrections, toutes mineures) se démarquent par leur capacité à produire du code fonctionnel dès le départ. ChatGPT a livré une application solide mais a nécessité des ajustements sur les tests et l'esthétique. Gemini a requis le plus de correctifs et d'améliorations, notamment sur des bugs visuels et des régressions de tests, qui montrent que la complexité des corrections IA peut dépasser celle du développement manuel.

La qualité du prompt, variable décisive. L'expérience avec Kimi K2.5 le démontre brillamment : la contrainte d'un seul prompt a forcé une formulation précise et structurée, qui s'est révélée être la stratégie la plus efficace. À l'inverse, des correctifs imprévisibles (Gemini, bordure coupée ; ChatGPT, tests cassés après ajout de couleur) montrent que même un prompt de qualité ne garantit pas l'absence de bugs. Le prompting est une compétence à part entière, au même titre que la programmation.

Comprendre ce qu'on n'a pas écrit, le vrai défi. Le risque principal identifié lors de cette phase n'est pas la qualité du code généré, souvent élevée, mais la perte de maîtrise technique qui peut en résulter. Un développeur qui intègre du code IA sans le comprendre en profondeur se retrouve incapable de le maintenir, de le déboguer ou de le faire évoluer sereinement. Les correctifs en cascade observés chez Gemini illustrent bien ce danger : modifier un module sans comprendre ses dépendances provoque des régressions difficiles à anticiper.

Un outil de développement, pas un substitut au développeur. En conclusion, les IA génératives de code sont de puissants outils d'accélération du développement lorsqu'ils sont utilisés par des développeurs capables de relire, comprendre et valider le code produit. Ils ne remplacent pas la compréhension algorithmique, la maîtrise des outils de

débogage, ni la culture des bonnes pratiques. La Phase 1, bien qu'elle ait produit une application plus modeste, a fourni des bases de compréhension indispensables pour exploiter intelligemment les résultats de la Phase 2. L'IA est à son maximum d'efficacité entre les mains d'un développeur compétent qui sait quoi demander, comment le demander, et comment évaluer ce qu'il reçoit.